

Real-Time Post-Decoding for Quantum Error Correction

Shipra Singh

Department of Computer Science
University of Colorado Boulder
Boulder, CO, USA

shipra.singh@colorado.edu

Narges Alavisamani

School of Computer Science
Georgia Institute of Technology
Atlanta, GA, USA

narges.alavisamani@gatech.edu

Ramin Ayanzadeh

Department of Computer Science
University of Colorado Boulder
Boulder, CO, USA

ayanzadeh@colorado.edu

Abstract—Real-time decoding is a critical bottleneck in Quantum Error Correction (QEC), where decoding must complete within a QEC cycle time (i.e., within $1\mu\text{s}$ in superconducting quantum systems). We observe that fast heuristic decoders such as Union-Find (UF) complete in tens of nanoseconds on average, leaving more than 95% of the time budget unused. Based on this observation, we propose *real-time post-decoding*, a new direction that uses the remaining time to refine decoding outcomes within the same cycle. We show that in most failed UF decodings, errors are localized: only a few pairs are incorrectly matched, while larger deviations are rare, forming a long-tail distribution. We introduce *TailCut*, the first realization of real-time post-decoding that uses the remaining time and available FPGA resources to perform bounded local re-matching, correcting dominant mismatches without exceeding the real-time budget or degrading performance. Our experiments show that TailCut corrects up to 63% of raw UF failures, corresponding to a logical error rate (LER) reduction of up to $2.7\times$, while preserving real-time execution. These results establish real-time post-decoding as a new direction for accurate real-time quantum error correction.

I. INTRODUCTION

Quantum hardware is inherently error-prone, making Quantum Error Correction (QEC) essential for reliable computation [1], [2]. QEC operates in repeated rounds where parity qubits are measured and decoded on a classical system to identify and correct errors on data qubits before they accumulate [1], [3]. Therefore, decoding must be completed within a strict hardware-defined cycle. For the surface code, the most popular QEC code, especially on superconducting hardware [1], [4]–[6], this window is approximately $1\mu\text{s}$ per round for superconducting devices [7], [8], which creates a strict timing budget for processing syndrome data. As code distance increases, the decoding problem becomes more complex, but the available time per cycle remains the same. This creates a fundamental trade-off between accuracy and latency in practical decoder design.

Existing decoding approaches struggle to simultaneously achieve high accuracy and real-time performance. Minimum-Weight Perfect Matching (MWPM) is the reference decoder for the surface code and provides near-optimal accuracy [7], [9], [10]. However, it exceeds the $1\mu\text{s}$ latency required for real-time operation: matching is done globally by taking the entire space-time syndrome graph at once, preventing the locality and parallelism required for efficient hardware

implementations [7], [10]–[13]. Fast heuristic decoders such as Union-Find (UF) meet real-time constraints but achieve lower accuracy [14]–[17]. Pre-decoding approaches, such as Promatch, apply a fast heuristic step that resolves simpler defect patterns, leaving a smaller residual syndrome for a slower global decoder. While this improves performance, cases where incorrect pairings are introduced early cannot be corrected later, leading to logical errors [8], [18]. As shown in Fig. 1(a), existing approaches either meet real-time constraints with reduced accuracy or achieve high accuracy at the cost of latency. This limitation stems from a common design choice: decoding is performed in a single pass, where once a matching is produced it is treated as final, leaving no opportunity for correction.

Fast decoders leave most of the time budget unused. We observe that state-of-the-art FPGA implementations of UF complete far earlier than the $1\mu\text{s}$ deadline, with an average latency of **35.5 ns**, consistent with prior work [17]. As a result, more than 95% of the available time budget remains unused. To our knowledge, prior work does not exploit this slack, and decoders terminate immediately upon producing a matching [7], [8], [17]. This unused time creates an opportunity to improve decoding accuracy without increasing latency.

Real-time post-decoding: a new direction for real-time decoding. We introduce *real-time post-decoding*, a design principle that refines decoding outcomes after the initial pass within the same cycle. Unlike existing decoders that terminate as soon as they produce a matching, post-decoding continues computation using the remaining time budget to correct residual errors. In contrast to pre-decoding, which simplifies the syndrome graph before decoding, post-decoding acts after the initial decoding by refining the final matching. This enables the decoder to correct suboptimal matching decisions that would otherwise cause logical failures, while remaining within the same real-time latency budget.

Decoding error distribution has a long tail. A key challenge in post-decoding is that the decoder does not know which parts of its output are incorrect. However, we observe that the number of mismatches between UF outcomes and MWPM solutions follow a *power-law* distribution, as illustrated in Fig. 1(b). Most UF matchings differ from the corresponding

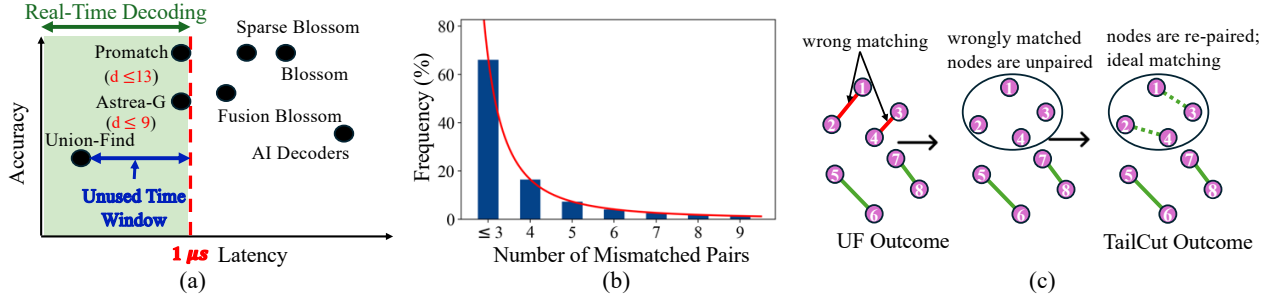


Fig. 1: Motivation for post-decoding. (a) UF often completes well below the $1 \mu s$ budget, leaving substantial slack. (b) Many UF failures differ from MWPM in only a few mismatched pairs. (c) TailCut uses the remaining time to re-match small local subsets of the initial solution.

MWPM matchings by only a few pairs, while some rare cases exhibit larger deviations, forming a long tail. This implies that decoding failures are dominated by only a small number of mismatches and can be corrected through targeted refinement.

TailCut: the first realization of real-time post-decoding.

We present *TailCut*, a post-decoding method that improves the accuracy of fast decoders such as UF while preserving real-time constraints. Given a UF outcome, we do not know whether it is correct or which subset of pairs, if any, is mismatched. Guided by the long-tail insight, TailCut explores combinations of k pairs, unpairs them, and re-matches the resulting $2k$ nodes (Fig. 1(c)). Each operation performs a local search that recomputes the best matching among the selected nodes using the same minimum-weight objective as MWPM.

II. TAILCUT DESIGN

TailCut executes after a first-pass UF decoder and starts from its decoding outcome. Instead of terminating immediately after UF produces a matching, TailCut uses the remaining real-time budget to refine that matching through bounded local repair. At a high level, TailCut identifies a small subset of matched pairs, temporarily unpairs them, evaluates alternative ways to reconnect their endpoints, and retains the repair with the lowest total matching weight.

Bounded Local Repair. For a first-pass UF decoder outcome of n pairs, TailCut selects a small subset of k pairs, temporarily unpairs them, and explores alternative re-pairings of the resulting $2k$ endpoints. For each candidate repair, TailCut recomputes the total matching weight and selects the alternative with the lowest weight.

Mapping to a Common Optimization Space. TailCut translates the UF outcomes into the MWPM space by replacing heuristic correction paths corresponding to the UF matching with minimum-weight paths obtained from a Dijkstra-based precomputed path table used by MWPM. This improves the baseline UF correction by replacing non-shortest paths with MWPM-consistent shortest paths while preserving the same matched endpoints. It also brings all candidates into the same optimization space as MWPM, so TailCut can compare repairs using a consistent minimum-weight objective.

Non-Regressive Refinement. TailCut applies a repair only if the repaired outcome has a lower total weight than the original UF outcome. Otherwise, the original matching is preserved. This makes the method non-regressive: post-decoding does not degrade the first-pass outcome.

Adaptive Execution Under Hardware Constraints. To budget FPGA resources, we use a conservative 75% usable fabric assumption for timing closure, based on AMD/Xilinx guidance [19], [20]. The LUT budget for TailCut is computed as the remaining capacity after placing the UF decoder, which, in our case, is an FPGA-based UF implementation called Helios [17], [21]. For instance, at $d = 9$, Helios uses 93,797 LUTs on the VCU129 FPGA, leaving the following LUT budget for TailCut:

$$2,586,000 \times 0.75 - 93,797 = 1,845,703 \text{ LUTs.}$$

TailCut evaluates repair candidates in parallel using dedicated units. Each unit computes the weight of one candidate, and a reduction tree selects the best repair. The degree of parallelism is determined by the remaining LUT budget, which limits the number of evaluation units that can be instantiated.

Let R denote the number of repair candidates that can be evaluated in parallel under the available LUT budget. If $N(n, k) \leq R$, all candidates are evaluated in one pass; otherwise, they are processed in batches of size at most R . The TailCut latency, measured in FPGA clock cycles, is

$$\text{cycles}(n, k) = \begin{cases} 2 + \lceil \log_2 k \rceil + \lceil \log_2 N \rceil, & \text{if } N \leq R, \\ 2 + \left\lceil \frac{N}{R} \right\rceil (\lceil \log_2 k \rceil + \lceil \log_2 R \rceil + 1), & \text{otherwise.} \end{cases}$$

The terms in the cycle model are interpreted as follows. The constant 2 accounts for fixed overhead. The term $\lceil \log_2 k \rceil$ corresponds to reduction over k pairs. If all candidates fit in one pass, $\lceil \log_2 N \rceil$ selects the best candidate. Otherwise, batching introduces an additional $\lceil \log_2 R \rceil + 1$ per batch.

TailCut then selects the largest feasible k , denoted $k_{\max}$, satisfying $\text{cycles}(n, k) \leq \text{remaining_cycles}$, where remaining_cycles is the number of FPGA clock cycles left after the first-pass UF decoder completes.

This makes the choice of k inherently adaptive: a larger k enables stronger repair but requires more resources and cycles.

The design integrates naturally with Helios, forming a two-stage pipeline in which UF provides fast decoding and TailCut uses remaining resources for bounded refinement.

III. EVALUATION METHODOLOGY

We evaluate TailCut on rotated surface-code memories with circuit-level depolarizing noise, for code distances $d = 3$ –17 and d rounds of syndrome extraction per trial. For each distance, we run 10^8 (100M) Monte Carlo trials. Syndromes are generated with Stim; PyMatching v2 is the MWPM reference decoder; UF serves as the first-pass decoder; TailCut is applied as bounded post-decoding.

To assess real-time viability, we model the FPGA-based UF decoder using the Helios implementation [17], [21] and assume a fixed end-to-end decoding budget of 1 μ s. We use the remaining cycle time and LUTs to determine the size of the repair that TailCut can achieve.

Our primary objective is to reduce the logical error rate (LER), which measures the probability of logical failure after decoding. We report LER across all configurations to evaluate the accuracy improvements achieved by post-decoding under a fixed real-time latency budget. We also examine, across code distances, the fraction of UF failures that TailCut corrects to quantify its effectiveness with respect to problem size.

IV. TAILCUT RESULTS

In this section, we evaluate the impact of TailCut on UF decoding accuracy.

A. Significant Improvement in LER

Table I summarizes LER for MWPM, UF, and UF + TailCut at $p = 10^{-3}$. We report results up to $d = 17$, as larger distances enter the rare-event regime, where there are insufficient failures for stable estimation. All results are obtained under identical bucket-based Monte Carlo sampling conditions, so differences in LER reflect decoder behavior rather than differences in sampling setup.

TABLE I: LER at selected code distances for MWPM, UF, and UF + TailCut at $p = 10^{-3}$. Values in parentheses report LER improvement relative to baseline UF.

d	MWPM	UF	UF + TailCut
3	9.9×10^{-4}	1.2×10^{-3}	8.0×10^{-4} (1.5 $\times$)
5	1.5×10^{-4}	2.3×10^{-4}	1.3×10^{-4} (1.76 $\times$)
7	2.1×10^{-5}	3.5×10^{-5}	1.6×10^{-5} (2.3 $\times$)
9	2.0×10^{-6}	4.0×10^{-6}	1.51×10^{-6} (2.6 $\times$)
11	1.2×10^{-7}	2.7×10^{-7}	1.0×10^{-7} (2.7 $\times$)
13	1.2×10^{-8}	5.0×10^{-8}	3.5×10^{-8} (1.42 $\times$)
15	4.1×10^{-9}	7.5×10^{-9}	7.3×10^{-9} (1.03 $\times$)
17	2.3×10^{-10}	4.2×10^{-10}	4.1×10^{-10} (1.02 $\times$)

Across the tested distances, TailCut substantially reduces UF LER, with the strongest gain at $d = 11$: UF drops from 2.7×10^{-7} to 1.0×10^{-7} , a 2.7 $\times$ improvement. Notably, for $d \leq 11$, UF + TailCut achieves lower LER than MWPM (for example, at $d = 11$, 1.0×10^{-7} vs 1.2×10^{-7}). This indicates that UF + TailCut can be a more refined strategy than MWPM alone in this regime. The key reason for this is degeneracy: many different matchings can have very similar costs, but not all have the same logical outcome. A single MWPM run returns one valid minimum-cost choice, while UF can land on a different degenerate solution. TailCut then refines that UF solution using the MWPM path table as a local cost guide, helping move to a better degenerate matching. Overall, the gains are strongest at small and moderate distances, where many UF failures result from only a few local mismatches. At higher distances, the improvement begins to decline because the maximum k that TailCut can accommodate within the same real-time budget is constrained, while the decoding problem size continues to grow.

B. Correction of a Large Fraction of UF Failures

Figure 2 shows the number of logical errors for UF and UF + TailCut. The reduction in failure count directly explains the LER improvement.

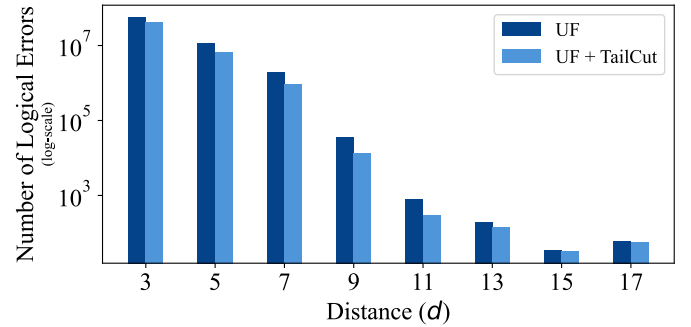


Fig. 2: Number of failed decodings for UF, and UF+TailCut across code distances(log-scaled). ($p = 10^{-3}$)

We observe that TailCut corrects a large fraction of UF failures across all distances, peaking at over 63.1% in the mid-distance regime. This behavior aligns with the long-tail insight: most UF failures differ from MWPM in only a few mismatched pairs, which TailCut can repair through bounded local re-matching. As a result, TailCut effectively converts a large portion of near-correct UF outputs into correct decodings.

V. RELATED WORK

TailCut relates to four main directions in surface-code decoding. First, matching-based decoders and accelerators improve MWPM but still focus on global decoding [9], [10], [12], [13], [22], whereas TailCut instead refines a fast heuristic output *after* the first pass using leftover cycle time for bounded local re-matching. Second, fast heuristic decoders such as UF prioritize real-time performance through local rules but often

suffer from reduced accuracy compared to MWPM [14]–[17], [23]. TailCut seeks to mitigate this loss by locally re-matching using the minimum-weight objective. Third, pre-decoding and hybrid approaches attempt to simplify the syndrome *before* decoding but often cannot revisit early local decisions [7], [8], [18], [24], [25], while TailCut operates *after* first-pass matching and specifically targets residual mistakes. Finally, learning- and optimization-based decoders can be highly accurate but struggle to guarantee deterministic 1 μ s execution [26]–[31].

Future work will quantitatively compare UF + TailCut with other hardware implementations of UF, MWPM, and pre-decoding frameworks such as Promatch and Astrea. Such comparisons will help clarify when post-decoding is most beneficial, how it complements existing pre-decoding and hardware-accelerated approaches, and whether similar bounded re-matching ideas can improve other fast decoders beyond UF.

VI. CONCLUSION

This paper introduces *real-time post-decoding*, a new direction for adaptive, scalable real-time quantum error correction systems that leverages unused cycle time to improve decoding accuracy while maintaining the same latency budget. We show that fast decoders such as UF leave substantial slack and that, in practice, their failures are dominated by a small number of localized mismatches in their decoding outcomes. We present *TailCut*, which exploits this structure via bounded local re-matching, achieving significant reductions in the LER while guaranteeing real-time execution under strict resource constraints.

REFERENCES

- [1] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Physical Review A*, vol. 86, no. 3, p. 032324, 2012.
- [2] Google Quantum AI, “Suppressing quantum errors by scaling a surface code logical qubit,” *Nature*, vol. 614, pp. 676–681, 2023.
- [3] A. G. Fowler, A. C. Whiteside, and L. C. L. Hollenberg, “Towards practical classical processing for the surface code,” *Physical Review Letters*, vol. 108, no. 18, p. 180501, 2012.
- [4] Y. Zhao, F. Jin, X. Pan *et al.*, “Realization of an error-correcting surface code with superconducting qubits,” *Nature*, vol. 601, pp. 531–536, 2022.
- [5] K. Takeda, A. Noiri, T. Nakajima, T. Kobayashi, and S. Tarucha, “Quantum error correction with silicon spin qubits,” *Nature*, vol. 608, pp. 682–686, 2022.
- [6] “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, no. 8052, pp. 920–926, 2025.
- [7] S. Vittal, P. Das, and M. Qureshi, “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect matching,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.
- [8] N. Alavisamani, S. Vittal, R. Ayanzadeh, P. Das, and M. Qureshi, “Promatch: Extending the reach of real-time quantum error correction with adaptive pre-decoding,” *arXiv preprint arXiv:2404.03136*, 2024.
- [9] A. G. Fowler, “Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $o(1)$ parallel time,” *arXiv preprint arXiv:1307.1740*, 2013.
- [10] O. Higgott, “Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–16, 2022.
- [11] V. Kolmogorov, “Blossom v: A new implementation of a minimum cost perfect matching algorithm,” *Mathematical Programming Computation*, vol. 1, no. 1, pp. 43–67, 2009.
- [12] O. Higgott and C. Gidney, “Sparse blossom: correcting a million errors per core second with minimum-weight matching,” *arXiv preprint arXiv:2303.15933*, 2023.
- [13] Y. Wu and L. Zhong, “Fusion blossom: Fast mwpm decoders for qec,” *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1, pp. 928–938, 2023.
- [14] N. Delfosse and N. H. Nickerson, “Almost-linear time decoding algorithm for topological codes,” *Quantum*, vol. 5, p. 595, 2021.
- [15] S. Huang, M. Newman, and K. R. Brown, “Fault-tolerant weighted union-find decoding on the toric code,” *Physical Review A*, vol. 102, no. 1, p. 012419, 2020.
- [16] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, “Afs: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 259–273.
- [17] N. Liyanage, Y. Wu, A. Deters, and L. Zhong, “Scalable quantum error correction for surface codes using fpga,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 916–927.
- [18] S. C. Smith, B. J. Brown, and S. D. Bartlett, “Local predecoder to reduce the bandwidth and latency of quantum error correction,” *Physical Review Applied*, vol. 19, no. 3, p. 034050, 2023.
- [19] AMD, “Ultrafast design methodology guide for amd fpgas and adaptive socs,” https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_1/ug949-vivado-design-methodology.pdf, 2022, uG949.
- [20] —, “Ultrafast design methodology timing closure quick reference guide,” <https://docs.amd.com/api/khub/documents/AJp~kDAavF8cxXvrO43cWQ/content>, 2022, uG1292.
- [21] N. Liyanage *et al.*, “Fpga-based distributed union-find decoder for surface codes,” *arXiv preprint arXiv:2406.08491*, 2024.
- [22] Y. Wu, N. Liyanage, and L. Zhong, “Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction,” *arXiv preprint arXiv:2502.14787*, 2025.
- [23] B. Barber, K. M. Barnes, T. Bialas, O. Buğdaycı, E. T. Campbell, N. I. Gillespie, K. Johar, R. Rajan, A. W. Richardson, L. Skoric *et al.*, “A real-time, scalable, fast and resource-efficient decoder for a quantum computer,” *Nature Electronics*, vol. 8, no. 1, pp. 84–91, 2025.
- [24] A. Knapen *et al.*, “Pinball: A cryogenic predecoder for quantum error correction decoding under circuit-level noise,” *arXiv preprint arXiv:2512.09807*, 2025.
- [25] C. Chamberland, L. Goncalves, P. Sivarajah, E. Peterson, and S. Grimberg, “Techniques for combining fast local decoders with global decoders under circuit-level noise,” *arXiv preprint arXiv:2208.01178*, 2022.
- [26] G. Torlai and R. G. Melko, “Neural decoder for topological codes,” *Physical Review Letters*, vol. 119, no. 3, p. 030501, 2017.
- [27] S. Krastanov and L. Jiang, “Deep neural network probabilistic decoder for stabilizer codes,” *Scientific Reports*, vol. 7, p. 11003, 2017.
- [28] C. Chamberland and P. Ronagh, “Deep neural decoders for near term fault-tolerant experiments,” *Quantum Science and Technology*, vol. 3, no. 4, p. 044002, 2018.
- [29] P. Baireuther, T. E. O’Brien, B. Tarasinski, and C. W. J. Beenakker, “Machine-learning-assisted correction of correlated qubit errors in a topological code,” *Quantum*, vol. 2, p. 48, 2018.
- [30] S. Varsamopoulos, B. Criger, and K. Bertels, “Decoding surface code with a distributed neural network based decoder,” *Quantum Machine Intelligence*, vol. 2, pp. 1–21, 2020.
- [31] L. Berent, L. Burgholzer, and R. Wille, “Decoding quantum color codes with maxsat,” *arXiv preprint arXiv:2303.14237*, 2023.